ОТЧЁТ

по лабораторной работе №20

«Контейнеризация, CI и эксплуатация Python-приложения»

Вариант 20. Планировщик задач

1. Цель работы

Научиться упаковывать приложение в образ, описывать запуск вместе с Redis и добавлять автоматическую проверку тестов в CI.

2. Постановка индивидуального задания

Вариант 20. Контейнеризировать планировщик задач и Redis. Добавить CI.

3. Краткие теоретические сведения

Образ Docker содержит интерпретатор и зависимости, поэтому программа запускается одинаково на разных машинах. docker-compose описывает два сервиса: redis и само приложение. Приложение получает адрес Redis через переменную REDIS_URL.

CI — проверка при каждой отправке кода. Файл GitHub Actions ставит Python 3.12, устанавливает зависимости и запускает pytest. Если тест падает, изменение не считается готовым.

4. Описание программы

Планировщик кладёт задания в список Redis: название, время и статус. Команда выполнения переводит в «выполнено» только те, чьё время уже наступило. Если REDIS_URL не задан, локальный запуск использует совместимый in-memory Redis, чтобы меню работало без демона.

Dockerfile собирает образ из requirements.txt. compose поднимает redis:7 и приложение с зависимостью от него. Тест добавляет раннюю и позднюю задачу и проверяет, что к полудню выполнена только ранняя.

5. Исходный код программы

Ниже приведён исходный код разработанных файлов.

Листинг — main.py

```
"""
Лабораторная работа №20. Вариант 20.
Планировщик задач. Рядом лежат Dockerfile, docker-compose и CI.
"""

import sys
```

```
from scheduler import Scheduler

if sys.platform == "win32":
    sys.stdout.reconfigure(encoding="utf-8")
    sys.stderr.reconfigure(encoding="utf-8")
    if not sys.stdin.isatty():
        sys.stdin.reconfigure(encoding="utf-8")



def read_line(prompt):
    if sys.stdin.isatty():
        return input(prompt).strip()
    raw = sys.stdin.readline()
    if raw == "":
        raise EOFError
    text = raw.rstrip("\r\n")
    print(f"{prompt}{text}")
    return text.strip()



def main():
    scheduler = Scheduler()
    print("Очередь задач в Redis. Если REDIS_URL не задан, используется локальный совместимый сервер.")
    while True:
        print()
        print("===== Планировщик =====")
        print("1. Добавить задачу")
        print("2. Показать очередь")
        print("3. Выполнить подошедшие задачи")
        print("0. Выход")
        choice = read_line("Выберите действие: ")
        if choice == "1":
            title = read_line("Название: ")
            run_at = read_line("Когда (ГГГГ-ММ-ДД ЧЧ:ММ): ")
            job = scheduler.add(title, run_at)
            print("Добавлено:", job)
        elif choice == "2":
            jobs = scheduler.list_jobs()
            if not jobs:
                print("Очередь пуста.")
            for index, job in enumerate(jobs, start=1):
                print(f"{index}. {job['run_at']} {job['title']} — {job['status']}")
        elif choice == "3":
            now = read_line("Текущее время (ГГГГ-ММ-ДД ЧЧ:ММ): ")
            done = scheduler.run_due(now)
            if not done:
                print("Подошедших задач нет.")
            for job in done:
                print("Выполнено:", job["title"])
        elif choice == "0":
            print("Работа программы завершена.")
            break
        else:
            print("Ошибка: выберите пункт от 0 до 3.")



if __name__ == "__main__":
    main()
```

Листинг — .github\workflows\ci.yml

```
name: ci
on:
  push:
  pull_request:
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: "3.12"
      - run: pip install -r requirements.txt
```

```
      - run: pytest
```

Листинг — Dockerfile

```
FROM python:3.12-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
CMD ["python", "main.py"]
```

Листинг — docker-compose.yml

```
services:
  redis:
    image: redis:7
    ports:
      - "6379:6379"
  scheduler:
    build: .
    environment:
      REDIS_URL: redis://redis:6379/0
    depends_on:
      - redis
    stdin_open: true
    tty: true
```

Листинг — requirements.txt

```
redis==8.1.0
fakeredis==2.38.0
pytest==9.1.1
```

Листинг — scheduler.py

```
"""Планировщик задач. Очередь хранится в Redis."""

import json
import os
from datetime import datetime

import fakeredis



def connect():
    url = os.environ.get("REDIS_URL", "")
    if url:
        import redis
        return redis.Redis.from_url(url, decode_responses=True)
    return fakeredis.FakeRedis(decode_responses=True)



class Scheduler:
    def __init__(self, client=None):
        self.client = client or connect()

    def add(self, title, run_at):
        job = {"title": title, "run_at": run_at, "status": "ожидает"}
        self.client.rpush("jobs", json.dumps(job, ensure_ascii=False))
        return job

    def list_jobs(self):
        raw = self.client.lrange("jobs", 0, -1)
        return [json.loads(item) for item in raw]

    def run_due(self, now=None):
        moment = now or datetime.now().strftime("%Y-%m-%d %H:%M")
        jobs = self.list_jobs()
        done = []
        self.client.delete("jobs")
        for job in jobs:
            if job["status"] == "ожидает" and job["run_at"] <= moment:
                job["status"] = "выполнено"
                done.append(job)
            self.client.rpush("jobs", json.dumps(job, ensure_ascii=False))
        return done
```

Листинг — tests\test_scheduler.py

```
import os
import sys

sys.path.insert(0, os.path.dirname(os.path.dirname(os.path.abspath(__file__))))

import fakeredis

from scheduler import Scheduler



def test_run_due_only():
    scheduler = Scheduler(fakeredis.FakeRedis(decode_responses=True))
    scheduler.add("Рано", "2026-09-27 10:00")
    scheduler.add("Позже", "2026-09-27 18:00")
    done = scheduler.run_due("2026-09-27 12:00")
    assert [job["title"] for job in done] == ["Рано"]
    titles = {job["title"]: job["status"] for job in scheduler.list_jobs()}
    assert titles["Рано"] == "выполнено"
    assert titles["Позже"] == "ожидает"
```

6. Примеры работы программы

Снимки показывают очередь планировщика и прогон теста.

```
C:\Users\Shmurdyakich\Desktop\pythonALL\python20> python main.py

Очередь задач в Redis. Если REDIS_URL не задан, используется локальный совместимый сервер.

===== Планировщик =====
1. Добавить задачу
2. Показать очередь
3. Выполнить подошедшие задачи
0. Выход
Выберите действие: 1
Название: Списать абонентскую плату
Когда (ГГГГ-ММ-ДД ЧЧ:ММ): 2026-09-27 09:00
Добавлено: {'title': 'Списать абонентскую плату', 'run_at': '2026-09-27 09:00', 'status': 'ожидает'}

===== Планировщик =====
1. Добавить задачу
2. Показать очередь
3. Выполнить подошедшие задачи
0. Выход
Выберите действие: 1
Название: Отчёт по тарифам
Когда (ГГГГ-ММ-ДД ЧЧ:ММ): 2026-09-27 18:00
Добавлено: {'title': 'Отчёт по тарифам', 'run_at': '2026-09-27 18:00', 'status': 'ожидает'}

===== Планировщик =====
1. Добавить задачу
2. Показать очередь
3. Выполнить подошедшие задачи
0. Выход
Выберите действие: 2
1. 2026-09-27 09:00 Списать абонентскую плату — ожидает
2. 2026-09-27 18:00 Отчёт по тарифам — ожидает

===== Планировщик =====
1. Добавить задачу
2. Показать очередь
3. Выполнить подошедшие задачи
0. Выход
Выберите действие: 3
Текущее время (ГГГГ-ММ-ДД ЧЧ:ММ): 2026-09-27 12:00
Выполнено: Списать абонентскую плату

===== Планировщик =====
1. Добавить задачу
2. Показать очередь
3. Выполнить подошедшие задачи
0. Выход
Выберите действие: 2
1. 2026-09-27 09:00 Списать абонентскую плату — выполнено
2. 2026-09-27 18:00 Отчёт по тарифам — ожидает

===== Планировщик =====
1. Добавить задачу
2. Показать очередь
3. Выполнить подошедшие задачи
0. Выход
Выберите действие: 0
Работа программы завершена.
```

Рисунок 1 – Добавление задач и выполнение тех, чьё время наступило

```
C:\Users\Shmurdyakich\Desktop\pythonALL\python20> python -m pytest -q

.                                                                        [100%]
1 passed in 0.21s
```

Рисунок 2 – Тест планировщика в том виде, как его запускает CI

7. Ответы на контрольные вопросы

1. Что описывает Dockerfile?

Базовый образ Python, установку зависимостей и команду запуска. По этому описанию собирается образ.

2. Зачем compose, если есть Dockerfile?

Приложению нужен ещё Redis. Compose поднимает оба сервиса и передаёт приложению адрес базы.

3. Как приложение находит Redis?

Через переменную REDIS_URL. В compose это redis://redis:6379/0, где redis — имя сервиса.

4. Что делает CI в этой работе?

На push и pull request ставит зависимости и запускает pytest. Падение теста видно до ручной проверки.

5. Какое правило времени проверено?

Задача выполняется, только если её время не позже заданного «сейчас». Более поздняя остаётся в статусе «ожидает».

8. Вывод

В лабораторной работе №20 планировщик варианта 20 хранит очередь в Redis, описан Dockerfile и docker-compose и проверяется в CI командой pytest. Локальный прогон показал, что к заданному времени выполняется только подошедшая задача.